UNIVERSITY OF PORTSMOUTH

COURSEWORK 1

SUBJECT-DATA MANAGEMENT

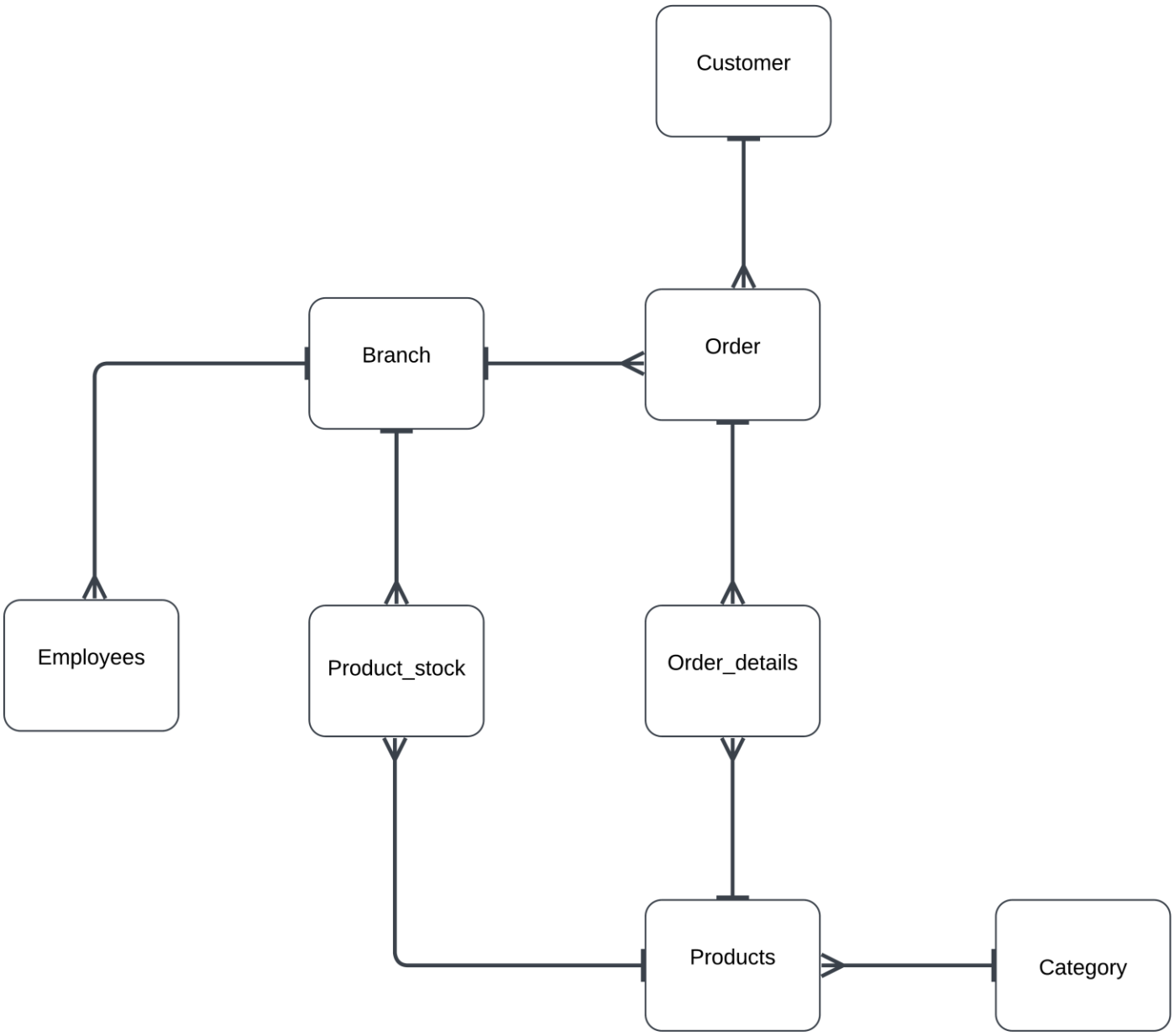
Assessment 1 - FM CLOTHING STORE case study

SUBMITTED BY:

- UP2290492
- UP2302988

Table of Contents

TASK 1: EERD


Customer
Branch
Order
Employees
Product_stock
Order_details
Products
Category

Task 2: **Reasoning and Assumptions:**

1. Since two tables (Products and Branch) have many-many relationships and PostgreSQL does not allow us to create an EERD of many-many relationships, we added the Product_stock table to the EERD. This table now has many-one relationships with both the Branch and Products tables. Additionally, we could not keep track of the stocks in each branch without this data, so we assumed the Products_stocks table to address these issues.
2. Initially we planned to include a manager's table to keep records of managers only but later we combined it with other staff to create an employees' table which includes the position of all the staff.
3. The business has six locations, which are included in our branch table: Portsmouth, Waterlooville, Fareham, Gosport, Havant, and Chichester. The manager of each branch is responsible for managing a specific branch. In the employee table employees have their unique employee_id, branch_id, and position, which makes it easier to identify the managers and other staff members employed by a particular branch.

Task 3: Data Dictionary:

1. **Customers Table:**

Customer					
Column's Name	**Primary keys**	**Data Types and Size**	**Domain and constraints**	**FK reference**	**Description (where non-obvious)**
customer_id	PK	INTEGER	PRIMARY KEY, UNIQUE, NOTNULL		Unique key identifying each customer
f_name		VARCHAR (100)	NOTNULL		The customer's first name
l_name		VARCHAR (100)	NOTNULL		The customer's last name
address		VARCHAR (100)	NOTNULL		Address of the customer
phone		VARCHAR (20)			The customer's contact number
email		VARCHAR (100)	NOTNULL		Email address of the customer

2. **Category Table:**

Category					
Column's Name	**Primary keys**	**Data Types and Size**	**Domain and constraints**	**FK reference**	**Description (where non-obvious)**
category_id	PK	INTEGER	PRIMARY KEY, UNIQUE, NOTNULL		Unique key tp identify each category

category_name		VARCHAR (100)	NOTNULL		Category's name

3. Product Table:

Products					
Column's Name	**Primary keys**	**Data Types and Size**	**Domain and constraints**	**FK reference**	**Description (where non-obvious)**
product_id	PK	INTEGER	PRIMARY KEY, UNIQUE, NOTNULL		Unique key identifying each product
name		VARCHAR (100)	NOTNULL		Name of the item
description		TEXT	NOTNULL		An explanation of the item
size		VARCHAR (50)	NOTNULL		Size of the product
category_id	FK	INTEGER	NOTNULL	Category	Category the product belongs to

4. Branch Table:

Branch					
Column's Name	**Primary keys**	**Data Types and Size**	**Domain and constraints**	**FK reference**	**Description (where non-obvious)**

branch_id	PK	INTEGER	PRIMARY KEY, UNIQUE NOTNULL		Unique key identifying each branch
branch	AK	VARCHAR (100)	UNIQUE, NOTNULL		Name of the branch

5. **Product_stock Table:**

Product_stock					
Column's Name	**Primary keys**	**Data Types and Size**	**Domain and constraints**	**FK reference**	**Description (where non-obvious)**
product_id	FK	INTEGER	NOTNULL	Product	Key that links to the primary key of the Product table
branch_id	FK	INTEGER	NOTNULL	Branch	Key that links to the primary key of the Branch table
quantity		INTEGER (100)	NOTNULL		Quantity of the product in stock

6. **Employee Table:**

Employees					
Column's Name	**Primary keys**	**Data Types and Size**	**Domain and constraints**	**FK reference**	**Description (where non-obvious)**

employee_id	PK	INTEGER	PRIMARYKEY, UNIQUE, NOT NULL		Each employee is identified by a unique key.
branch_id	FK	INTEGER	NOT NULL	Branch	Key that links to the primary key of the Branch table
f_name		VARCHAR (100)	NOT NULL		Employee's first name
l_name		VARCHAR (100)	NOT NULL		Employee's last name
email		VARCHAR (100)			Employee's email id
position		VARCHAR (100)	NOT NULL		Employee's designation

7. Order Table:

Orders					
Column's Name	**Primary keys**	**Data Types and Size**	**Domain and constraints**	**FK reference**	**Description (where non-obvious)**
order_id	PK	INTEGER	PRIMARY KEY, UNIQUE, NOTNULL		Unique key identifying each order placed
customer_id	FK	INTEGER	NOTNULL	Customer	Customer who placed the order
branch_id	FK	INTEGER	NOTNULL	Branch	Branch in which the order is placed
order_date		DATE	NOTNULL		The date of order placement
total_amount		DECIMAL (10,2)	NOTNULL		The total order amount

8. **Order_details Table:**

Orders_details					
Column's Name	**Primary keys**	**Data Types and Size**	**Domain and constraints**	**FK reference**	**Description (where non-obvious)**
order_detail_id	PK	INTEGER	NOTNULL		Unique key identifying each order detail
order_id	FK	INTEGER	NOTNULL	Orders	Key that links to the primary key of the Orders table
product_id	FK	INTEGER	NOTNULL	Products	Key that links to the primary key of the Product table
quantity		INTEGER	NOTNULL		Quantity of the product ordered
price		DECIMAL (10,2)			product's price

Task 4: Five PostgreSQL Queries.

1. **Query to obtain order count and total sales per branch.**

```
up2302988=# \c fm_store
You are now connected to database "fm_store" as user "up2302988".
fm_store=# -- 1. Query to Get Total Sales per Branch with Order Count

SELECT
    b.branch_name ,
    COUNT(o.order_id) AS Total_Orders,
    SUM(o.total_amount) AS Total_Sales
FROM
    Orders o
JOIN
    Branch b ON o.branch_id = b.branch_id
GROUP BY
    b.branch_name
ORDER BY
    Total_Sales DESC;
      branch_name       | total_orders | total_sales
------------------------+--------------+-------------
 Havant                 |            8 |     1283.80
 Chichester             |            8 |     1194.05
 Portsmouth Head Office |            9 |     1116.95
 Fareham                |            8 |     1111.60
 Waterlooville          |            9 |     1077.50
 Gosport                |            8 |      698.15
(6 rows)
```

Query Explanation:

- **SELECT** b.branch AS Branch_Name: This picks the branch name from the Branch table.
  - COUNT(o.order_id) AS Total_Orders: This counts the number of orders for each branch then labeled the result as Total_Orders.
  - SUM(o.amount) AS Total_Sales: This sums up the total sales amount for each branch then labeled as Total_Sales.
- FROM Orders o: This explains the Orders table as the main table for the query, using the alias o.
- **JOIN Branch b ON** o.branch_id **=** b.branch_id: This joins the Orders table with the Branch table by matching branch_id in both tables, so we can get branch information for each order.
- GROUP BY b.branch_name : It is grouped by the branch name.
- **ORDER BY Total_Sales** DESC: This sorts the results in descending order by total sales, so the branch with the highest sales appears first.

Query Use Case:

This query provides a ranked list of branches with the highest total sales and the number of orders for each branch. It helps businesses identify which branches are performing best in terms of sales.

2. Query to list the top 5 most popular products by the total quantity sold.

```
fm_store=# -- "List the top 5 most popular products by the total quantity sold."

SELECT
    pd.name AS Product_Name,
    SUM(od.quantity) AS quantity_sold
FROM
    Order_Details od
JOIN
    Products pd ON od.product_id = pd.product_id
GROUP BY
    pd.product_id
ORDER BY
    quantity_sold DESC
LIMIT 5;
  product_name  | quantity_sold
----------------+---------------
 Men T-Shirt    |            12
 Men Sneakers   |            11
 Leather Wallet |            11
 Running Shoes  |            11
 Formal Suit    |            11
(5 rows)
```

Query Explanation:

- **SELECT** pd.name AS Product_Name: Gets the product name from the Products table, labeled as Product_Name.
  - SUM (od.quantity) AS quantity_sold: It sums up the quantity sold for each product then it is labeled as quantity_sold.
- FROM Order_Details od: Uses the Order_Details table as the main table, with the alias od.
- JOIN Products pd ON od.product_id = pd.product_id: It joins the table Order_Details with the Products table based on the product_id.
- GROUP BY pd.product_id: It groups the results by product ‘s id and the quantity sold is calculated per product.
- ORDER BY quantity_sold DESC: The results are sorted from top to lowest by the total quantity sold.
- **LIMIT 5**: only displays the top five goods in terms of amount sold.

Query Use Case: This query helps store managers, sales analysts, or inventory managers identify the top-selling products. It provides insights into product performance, which can be used for inventory management, stocking decisions, and sales forecasting.

3. **Query to retrieve top 5 customer by total spending.**

```
fm_store=# -- 3. Query to Retrieve Top 5 Customers by Total Spending
-- This query finds the top 5 customers based on their total spending, helping the business identify and possibly reward loyal,
-- high-spending customers.


SELECT
    cu.f_name || ' ' || cu.l_name AS Customer_Name,
    SUM(o.total_amount) AS total_spending
FROM
    Orders o
JOIN
    Customers cu ON o.customer_id = cu.customer_id
GROUP BY
    cu.customer_id
ORDER BY
    total_spending DESC
LIMIT 5;
  customer_name  | total_spending
-----------------+----------------
 Mary Anderson   |         386.05
 Robert Taylor   |         366.25
 Daniel Wilson   |         360.70
 James Martin    |         355.75
 Angela Martinez |         330.50
(5 rows)
```

Query Explanation:

- SELECT cu.f_name || ' ' || cu.l_name AS Customer_Name: Combines the first name (f_name) and last name (l_name) of customers into a full name, labeled as "Customer_Name."
- SUM(o.total_amount) AS total_spending: It calculates the total amount spent by each customer, labeled as total_spending.
- FROM Orders o: Uses the Orders table as the main table, with the alias o.
- JOIN Customers cu ON o.customer_id = cu.customer_id: Joins the Orders table with the Customers table according to the customer's id.
- **GROUP BY** cu.customer_id: It groups the results by the customer and the total spending is calculated as per customer.
- ORDER BY total_spending DESC: Sorts the results by total spending, from highest spending to lowest spending.
- LIMIT 5: It limits its results to the top five clients who spend the most money overall.

Query Use Case:

This query is useful for the marketing team to analyze customer value, identify high-spending clients, and make data-driven decisions across various operational areas. They can customize advertisements based on the high spending customers.

4. Query to check stock levels and identify low stock product.

```
fm_store=# -- 4. Query to Check Stock Levels and Identify Low-Stock Products
-- This query identifies products with low stock levels (e.g., less than 10 items in stock). The results include the product
-- name, category, and current stock count, which is useful for inventory management.
SELECT
    ps.product_id,
    pd.name AS product_name,
    ps.branch_id,
    b.branch_name ,
    ps.quantity
FROM
    Product_stocks ps
JOIN
    Products pd ON ps.product_id = pd.product_id
JOIN
    Branch b ON ps.branch_id = b.branch_id
WHERE
    ps.quantity < 30
ORDER BY
    ps.quantity ASC;
 product_id |  product_name  | branch_id |      branch_name       | quantity
------------+----------------+-----------+------------------------+----------
          7 | Yoga Pants     |         3 | Fareham                |       20
          3 | Kids Hoodie    |         2 | Waterlooville          |       20
          8 | Formal Suit    |         6 | Chichester             |       20
          1 | Men T-Shirt    |         5 | Havant                 |       20
          4 | Leather Wallet |         6 | Chichester             |       25
          2 | Women Blouse   |         1 | Portsmouth Head Office |       25
          7 | Yoga Pants     |         6 | Chichester             |       25
(7 rows)
```

Query Explanation:

- SELECT ps.product_id: Gets the product_id of product from the ProductStock table.
  - pd.name AS product_name: Gets the product name from the Products table, labeled as product_name.
  - ps.branch_id: Gets the id of branch from Product_stocks table.
  - b.branch_name: Gets the branch name from the Branch table.
  - ps.quantity: Shows the quantity of the product available at the branch.
- FROM ProductStock ps: Uses the ProductStock table as the main table, with the alias ps.
- JOIN Products pd ON ps.product_id = pd.product_id: It joins the ProductStock table with the Products table according to the product_id .
- **JOIN Branch b ON** ps.branch_id **=** b.branch_id: It joins the Product_stock table with the Branch table based on the branch ID.
- **WHERE** ps.quantity **< 30**: The results are filtered to display only items with a quantity under 30.
- **ORDER BY** ps.quantity ASC: The results are sorted in ascending order by quantity.

Query Use Case:

This query helps inventory managers or supply chain departments to identify products that are low in stock across different branches. They can use this information to reorder products before they run out, ensuring branches stay stocked with popular items.

5. Query to get monthly revenue and order count for the past year.

```
fm_store=# -- 5. Query to Get Monthly Revenue and Order Count for the Past Year
SELECT
    DATE_TRUNC('month', order_date) AS Month,
    COUNT(order_id) AS Total_Orders,
    SUM(total_amount) AS Monthly_Revenue
FROM
    Orders
WHERE
    order_date >= CURRENT_DATE - INTERVAL '1 year'
GROUP BY
    Month
ORDER BY
    Month;
         month          | total_orders | monthly_revenue
------------------------+--------------+-----------------
 2024-01-01 00:00:00+00 |            2 |          225.75
 2024-02-01 00:00:00+00 |            2 |          170.75
 2024-03-01 00:00:00+00 |            3 |          335.90
 2024-04-01 00:00:00+00 |            4 |          486.45
 2024-05-01 00:00:00+00 |            4 |          565.90
 2024-06-01 00:00:00+00 |            4 |          376.65
 2024-07-01 00:00:00+00 |            4 |          591.45
 2024-08-01 00:00:00+00 |            4 |          551.65
 2024-09-01 00:00:00+00 |            4 |          556.35
 2024-10-01 00:00:00+00 |            4 |          520.90
 2024-11-01 00:00:00+00 |            4 |          506.45
 2024-12-01 00:00:00+00 |            4 |          591.55
 2025-01-01 00:00:00+00 |            4 |          556.65
 2025-02-01 00:00:00+00 |            3 |          445.65
(14 rows)
```

Query Explanation:

- SELECT DATE_TRUNC ('month', order_date) AS Month: This trims the order_date to the first day of the month and labels it as Month.
  - COUNT (order_id) AS Total_Orders: This counts the number of orders for each month.
  - SUM (total_amount) AS Monthly_Revenue: This calculates the total revenue (the sum of the column total_amount) for each month.
- FROM Orders: Uses the Orders table.
- **WHERE order_date >= CURRENT_DATE -** INTERVAL '1 year': It filters orders to include only those data which were made in the last year.
- **GROUP BY Month**: Groups the data by month, so each row represents a different month.
- ORDER BY Month: Sorts the results in ascending order by month.

Query Use Case:

This query is useful for tracking monthly order activity and revenue over the past year. It helps businesses analyze trends, monitor monthly performance, and plan for future operations.

Task 6: Reflective Report

We carefully planned and prepared for this project. We first met at the library to talk about the ERD diagram. We faced several obstacles at that time, but we collaborated to overcome them and produced the ERD. We then distributed the tasks across the team to increase efficiency. My responsibility was to develop the database in accordance with our EERD after finishing the ERD. I created the database, went over it with the group, and added the information. Since the relationship was unclear when the code was being created, we talked about what should be included and what should be left out of the EERD. As a result, we had to rebuild the EERD before finalizing it. Creating dummy data was the next step. A portion of the data was generated manually, while others were generated using data-generating websites. We added the information. Additionally, we successfully created a database based on EERDD on my PostgreSQL work bench by running some sample code there. Following success, I executed my code in my virtual machine (VM), built a database, and then entered the data into the table. Since it was my first time using VM, I needed assistance understanding it completely, therefore I used the SQL shortcut file that was uploaded to Moodle. After running several queries on the virtual machine (VM) and finding that it was functioning properly, we chose five queries that we believed would be helpful for business purposes. Now it was time to create a report, which was the portion for my team since we split our assignment previously. My team member prepared a report, and we examined and modified it. In this project, we discovered the value of careful preparation, teamwork, flexibility, and creativity through the course of this project.

Reference:

- SQL Tutorial. (n.d.). W3Schools. https://www.w3schools.com/sql/

- Beaulieu, A. (2009). *Learning SQL* (2nd ed.). O'Reilly Media.

  https://moodle.port.ac.uk/pluginfile.php/4959777/mod_resource/content/1/Learning%20SQL%2C%202nd%20Edition%20E%20Book.pdf
- Matthew, O. (2024, November 4 - 10). Foreign Keys and Joins - Answers [Slides]. https://moodle.port.ac.uk/pluginfile.php/4959884/mod_resource/content/1/foreign_keys_answers.sql